Deep Learning Practical 3: Model answers

Encoders, bottlenecks and latent distributions

2 Colour prediction from a greyscale input

1. How many pixel intensity values are in one 32×32 greyscale image? How many are in its RGB target?

A greyscale input contains $1 \times 32 \times 32 = 1,024$ pixel intensity values, so its flattened vector has length 1,024. Its RGB target contains $3 \times 32 \times 32 = 3,072$ channel intensity values, so its flattened vector has length 3,072. Both images have 1,024 pixels; the difference is the number of channels per pixel.

2. Find two RGB colours with the same grey value. Start from $(0.5, 0.5, 0.5)$ and add $(0.0587, -0.0299, 0)$. What information has the conversion discarded?

The colours $(0.5, 0.5, 0.5)$ and $(0.5587, 0.4701, 0.5)$ both have grey value 0.5, because the change is $0.299(0.0587) - 0.587(0.0299) = 0$. Converting three channels to one retains a weighted brightness value but removes two degrees of freedom, so the original hue and saturation cannot in general be recovered from grey alone.

3 Encode an image into a small vector

1. Trace x through g, E_ϕ and D_θ to $\hat{x}$. Which weights and biases are learned during training?

Starting with a colour image, we compute $c = g(x)$, encode it as $z = E_\phi(c)$ and decode it as $\hat{x} = D_\theta(z)$. Training adjusts the encoder weights and biases ϕ and decoder weights and biases θ together, while the greyscale conversion g stays fixed. Neither c nor z is a parameter stored by the model: both are calculated from the current image, with z also depending on the current encoder parameters.

2. Count the encoder and decoder weights and biases for $d = 4$ and $d = 64$. Why does the parameter count grow much less than sixteenfold?

Including biases, the four affine layers contain

$$(1024 \times 256 + 256) + (256d + d) + (d \times 256 + 256) + (256 \times 3072 + 3072) = 1,052,160 + 513d$$

parameters. Thus $d = 4$ gives 1,054,212 and $d = 64$ gives 1,084,992. Only the two layers adjacent to the bottleneck grow with d. Most parameters remain in the large input and output layers, with 789,504 in the final layer alone, which explains the relatively small increase in the total.

3. With `skip=False`, remove both ReLUs and write the network as $\hat{x} = Ac + b$. Express A and b using the layer weights and biases. How does d bound $\mathrm{rank}(A)$?

Writing c as a flattened vector, composing the four layers without ReLUs gives $\hat{x} = Ac + b$, where

$$\begin{aligned} A &= W_4 W_3 W_2 W_1, \\ b &= W_4 W_3 W_2 b_1 + W_4 W_3 b_2 + W_4 b_3 + b_4. \end{aligned}$$

Every change in the output must pass through the d-dimensional bottleneck, so $\mathrm{rank}(A) \leq d$. This applies to the plain encoder–decoder. A direct greyscale skip would add another route around that bottleneck.

4 Train the bottlenecks and add a skip connection

1. Compare the sail's edge, vehicle windows and animal faces. Why can the four-value skip model produce much more detail than the four-value plain model?

Because the grey input already contains the sail's edge, window boundaries and facial detail at the correct pixel positions, the skip can copy them straight to the output, leaving the decoder to add an RGB correction. Without this route, the model must reconstruct all that detail from just four numbers, which produces much smoother shapes.

2. For two images, use the $d = 4$ skip model to zero their latent vectors, then swap their original vectors. Keep each grey input fixed. How do colour, brightness and outlines change?

Setting $z = 0$ gives $r(c) + D_\theta(0)$, where $r(c)$ repeats the grey image across RGB. The grey structure still reaches the output, but the correction need not vanish because the trained decoder has biases. Swapping codes changes this correction while keeping the photograph on the skip path fixed, so its outlines usually remain visible even as colour and brightness change. Nothing constrains the correction to preserve luminance exactly, and a code from another photograph may give implausible colours because that code–image pairing differs from the pairs used during training.

3. Train plain and skip models with $d = 16$, keeping other settings fixed. Compare with $d = 4$ and $d = 64$. Does sharper output imply more shape information in the latent vector?

With $d = 16$, either version has 1,060,368 parameters. A CPU comparison with weight seed 3801, batch seed 3802, batches of 64 and 6,000 updates at learning rate 0.001 gives validation MSE 0.019957 without the skip and 0.005798 with it. Without the skip, the sail and animal faces remain blurred. With the skip, their outlines are clear even though some colours remain muted. Sharpness alone cannot tell us how much shape information the latent vector contains. In the skip model, sharp outlines can come directly from c even when the decoder contributes only broad colour regions. Holding c fixed while changing z isolates what the latent path actually changes.

Training length and colour uncertainty

Retrain one model with a different weight seed or more updates. Compare validation RGB MSE and colours. Then train with mean absolute error (L_1). For one pixel channel, what statistic of the target intensity given the grey input minimises expected absolute error?

For the plain $d = 64$ model, increasing training from 6,000 to 15,000 updates reduces validation MSE from 0.013175 to 0.012929. Most of its improvement occurs earlier: at 1,500 updates the error is 0.016326. Longer training refines the prediction, but it cannot determine a colour that the grey input leaves ambiguous. In particular, an accurate average of several plausible colours can still look muted.

For one uncertain colour value, a conditional median minimises expected absolute error, while the conditional mean minimises expected squared error. Replacing MSE by L_1 therefore changes the preferred compromise, but still produces a single colour estimate rather than several plausible alternatives.

5 Interpolate the two paths separately

1. At $a = 1$, which rows match their model's prediction for the second image in the pair? What causes overlapping outlines in the "Blend both paths" row?

At $a = 1$, rows 1, 2 and 4 use the second image's latent code and, where a skip is present, its grey input too. They therefore finish at their model's ordinary prediction for that image. Row 3 combines the second code with the first image's grey input, so it need not match either ordinary prediction. In row 2, the overlapping outlines arise from explicitly blending the two grey photographs on the skip path, without the decoder needing to invent an intermediate object shape.

2. Compare $D_\theta((1-a)z_0 + az_1)$ with $(1-a)D_\theta(z_0) + aD_\theta(z_1)$ before clipping. Give a condition under which they agree for every $a \in [0,1]$.

If $D(z) = Az + b$ is affine, then

$$D((1-a)z_0 + az_1) = (1-a)D(z_0) + aD(z_1).$$

A ReLU decoder is affine within each activation region, so the equality also holds when the whole segment stays in one such region. Crossing a ReLU boundary can change the affine map and break the equality. Clipping the displayed output can introduce a further difference from these raw decoder calculations.

3. Repeat the interpolation plot with both $d = 4$ models. In each row, which changes come from the decoder and which from the grey skip input?

Four latent values chiefly change broad regions of colour and brightness. In the first row, all changes come from the decoder. In rows 3 and 4, the grey structure stays fixed, so changes along each row again come from the decoded correction. Row 2 also changes the grey input, which explains its moving or overlapping outlines without requiring the latent vector to encode a new object shape. Intermediate codes may produce smooth transitions while still decoding to implausible photographs.

6 What distribution do the latent vectors follow?

1. Plot the first two latent coordinates across training images. What suggests dependence? Can the plot establish independence? Would standardising each coordinate remove dependence?

A scatter plot and sample correlations can reveal dependence:

```
with torch.no_grad():
    z = plain4.encoder(grey(images[:4096]).to(device)).cpu()
plt.scatter(z[:, 0], z[:, 1], s=2, alpha=.2)
print(torch.corrcoef(z.T))
```

For the first two coordinates of the four-value encoder retrained with the sheet's seeds and 6,000 updates on CPU, the sample correlation is approximately -0.267. This observed association is evidence against treating the coordinates as independent. A finite scatter plot cannot establish independence, even when no pattern is visible. Subtracting a mean and dividing by a standard deviation changes the units of each coordinate, but preserves their correlation and any nonlinear dependence. Conversely, zero correlation would not establish independence, since dependent variables can still be uncorrelated.

2. Does the loss encourage $q_\phi(z)$ to match a Gaussian? What information does it encourage the codes to retain?

There is no term requiring the codes to match a Gaussian. The objective only penalises the difference between the decoded prediction and the RGB target, so it encourages codes from which the decoder can predict colour. Even if individual coordinate histograms look Gaussian, their joint distribution may have a different shape.

7 Try a Gaussian in the latent space

1. Compare Gaussian samples with encoded-image predictions for each d. For $d = 4$, does similarity in blur and colour imply realistic object structure?

Both four-value rows contain smooth patches of colour with little object detail, which makes the Gaussian samples look superficially similar to the reconstructions. That resemblance tells us little about whether the samples depict recognisable photographs, since the reconstructions themselves have lost most of the detail. With 64 values, the reconstructions retain more structure, making the implausible arrangements in the sampled row easier to see.

2. Replace the fitted samples with `torch.randn(8, d)`. How might their means, variances and correlations mismatch the training codes? Do mean and covariance fully specify any distribution?

Standard normal samples assume zero means, unit variances and independent coordinates. Encoded images need not satisfy any of those assumptions, so these samples can land far from the codes encountered in training. Fitting a full Gaussian corrects the estimated means, scales and correlations, but it still misses properties such as multiple modes or a curved concentration of probability. Mean and covariance do not determine an arbitrary distribution.

3. Decode eight training-image codes instead of Gaussian samples. Which model predictions does this reproduce? What assumption does Gaussian sampling add about the latent distribution?

Selecting an encoded training code and decoding it reproduces the model's prediction for that training input. Sampling a continuous Gaussian instead assumes a particular distribution between and beyond those observed codes. The reconstruction objective gives no guarantee that such new codes will decode to realistic images.

8 What happens when we sample the skip path?

1. In the second row, where does the recognisable object come from? Does a random z make this a sample of a new object?

The recognisable object comes from the real grey photograph on the skip path. A random code changes its colour correction, so this is a new treatment of an existing object rather than a newly generated object.

2. The two sampled paths have a joint Gaussian distribution. Does that enforce the nonlinear relation $z = E_\phi(c)$ seen during training? Check the sampled z against `skipped4.encoder(new_grey)`.

A joint Gaussian captures how pixels and codes vary together through their covariance, but this does not require a sampled code to equal the encoder's output for its sampled image. During training, every pair satisfied $z = E_\phi(c)$ exactly. We can measure how far a sampled pair departs from that relationship:

```
with torch.no_grad():
    discrepancy = (new_z-skipped4.encoder(new_grey)).square().mean()
print(discrepancy.item())
```

The discrepancy is generally nonzero because matching means and covariances does not enforce the encoder's nonlinear mapping.

3. Replace the joint Gaussian's covariance by its diagonal, keeping its mean fixed. How does this affect spatial structure in the sampled grey images and RGB outputs?

Keeping only the diagonal preserves each fitted marginal Gaussian but removes covariance between pixels, between latent coordinates, and between the latent and pixel coordinates. The full Gaussian can produce some smooth spatial variation, whereas independent pixels produce fine noise around the per-pixel means. Neither construction captures the arrangements of parts that make a coherent photograph.

4. What extra distributions or training objectives would we need to generate photographs without supplying a real grey image? What would we need to generate several plausible colourisations of one fixed grey image?

To generate without a real grey photograph, we need to model all the inputs to the decoder. One possibility is a generator for grey images followed by a conditional colour model. Another is a generator trained directly on RGB images. Both require an objective that learns a distribution of images, rather than reconstruction of a supplied input alone.

For several colourisations of one fixed grey image, we need a conditional model $p_\theta(x \mid c)$ that can use fresh noise to select among plausible colours. A deterministic encoder and decoder with an MSE objective instead return one estimate for that input.